Chapter 12: A Web Application with Mongita

We have used Mongita interactively and seen how document operations fit behind `database.py`. Now we will follow a complete application through that layer. The browser still displays pets and owners. Forms still create and change records. What has changed is how the application saves those records and keeps their relationships meaningful.

A document database stores records with named fields and can include nested records and lists. (1) That gives us choices about how to organize related data. This application keeps pets and owners in separate collections, with an owner identifier in each pet document. We will follow that reference from an HTML form to the stored document and back.

The Application We Are Running

There are three main parts:

Part	Responsibility
`app.py`	Receive requests, call the database layer, select a response
`templates/`	Display dictionaries and build forms and links
`database.py`	Validate values, find related records, read and write documents

Mongita runs inside Python and implements a subset of the MongoDB/PyMongo application programming interface (API). PyMongo is the Python driver for MongoDB. Here, Mongita stores documents on local disk; no MongoDB server is needed. (2)

The same separation we used for relational storage still applies. A route asks for a pet by identifier. The database layer decides how that request becomes a document query. The template receives a dictionary and does not need to know how the database stored it.

Integrity Rules in the Database Layer

Data integrity means keeping stored data valid and its relationships consistent. In the relational version, we declared constraints and let SQLite enforce them. With this Mongita version, we implement those integrity rules ourselves in `database.py`.

Integrity rule	Relational version	Mongita database layer
Pet name and type must be present and nonblank	`NOT NULL` and `CHECK` constraints	Validate required text
Age cannot be negative	`CHECK (age >= 0)`	Reject negative ages before saving
A pet must have an existing owner	Required foreign key	Look up the owner before saving
An owner with pets cannot be deleted	Foreign key with `ON DELETE RESTRICT`	Check for pets before deleting the owner

The rules have not changed. Responsibility for enforcing them has moved from the relational database to our Python database layer. Both create and update use the same validation helper, so they apply the same rules. The routes continue to call the layer without implementing those checks themselves.

Start the App

Use Python 3.10 or newer. **Try This** marks commands or actions to perform. Other code blocks show reference excerpts or example records. The browser activities below build on one another.

Try This: From the repository root, create an environment and install the supplied requirements:

```
cd chapter-12-mongo-app/code
python3 -m venv ~/venvs/advanced-database-mongo-app
source ~/venvs/advanced-database-mongo-app/bin/activate
python -m pip install -r requirements.txt
flask --app app run --debug
```

These activation commands use a Bash-style shell. In Windows PowerShell, activate the environment with `~\venvs\advanced-database-mongo-app\Scripts\Activate.ps1` instead. Flask's `--app` option selects the application module; `run` starts the development server. (3) Open the address printed in the terminal.

At startup, `app.py` calls:

```
database.setup_database("pets")
```

This selects the `pets` database and opens its `owners` and `pets` collections. It does not erase existing records or insert sample owners. An unused store starts empty.

The local files live in `code/data/mongita`. The path is calculated from `database.py`, so launching from a different directory does not silently select another store. Keep the generated `data/` folder out of Git. This directory has its own store, separate from the introductory animal-record example and the Pets copy used there.

Create an Owner Before a Pet

Try This: Follow **Manage Owners**, create an owner named Alex, and fill in a city and type of home. Then create another owner named Blair. Return to the pet list and create Casey, a three-year-old dog belonging to Alex.

The owner form submits a name and optional descriptive fields. The pet form submits the owner's identifier, not the owner's name. Two owners can have the same name; their identifiers still distinguish them.

Here are illustrative stored documents. The identifiers stand for particular records, not values to copy into the form:

```
# owners collection
{"_id": ObjectId("67d8c61b5180a31695e90745"),
 "name": "Alex", "city": "Kent", "type_of_home": "house"}

# pets collection
{"_id": ObjectId("67d8c61b5180a31695e90744"),
 "name": "Casey", "type": "dog", "age": 3,
 "owner_id": ObjectId("67d8c61b5180a31695e90745")}
```

The pet's `owner_id` matches the owner's `_id`. We do not copy the owner's city into every pet. Updating Alex's address therefore changes one owner record.

Why use a reference here when the grocery-purchase example embedded its line items? An owner exists independently of a particular pet and can be shared by several pets. A purchase line item belongs to one purchase. Embedding and referencing serve different relationships; MongoDB's modeling guidance ties the choice to how related data is read, updated, and shared. (4)

Identifiers Across the Boundary

The database stores `ObjectId` values. The application uses their 24-character hexadecimal string representations. The constructor converts that text back into an `ObjectId`. (5)

Location	Identifier representation
Stored pet `_id`	`ObjectId`
Stored pet `owner_id`	`ObjectId` referring to an owner
Dictionary returned to a route	String in `id` or `owner_id`
Form value or address	Text
Lookup filter inside the layer	Converted `ObjectId`

The conversion happens in one helper:

```
def _to_object_id(value, field_name="id"):
    try:
        return ObjectId(str(value))
    except (InvalidId, TypeError) as exc:
        raise ValueError(
            f"{field_name} must be a valid ObjectId string."
        ) from exc
```

This excerpt wraps the error message to fit the page. Its behavior matches the supplied function. A valid identifier string is not proof that a document exists. The layer checks those two things separately.

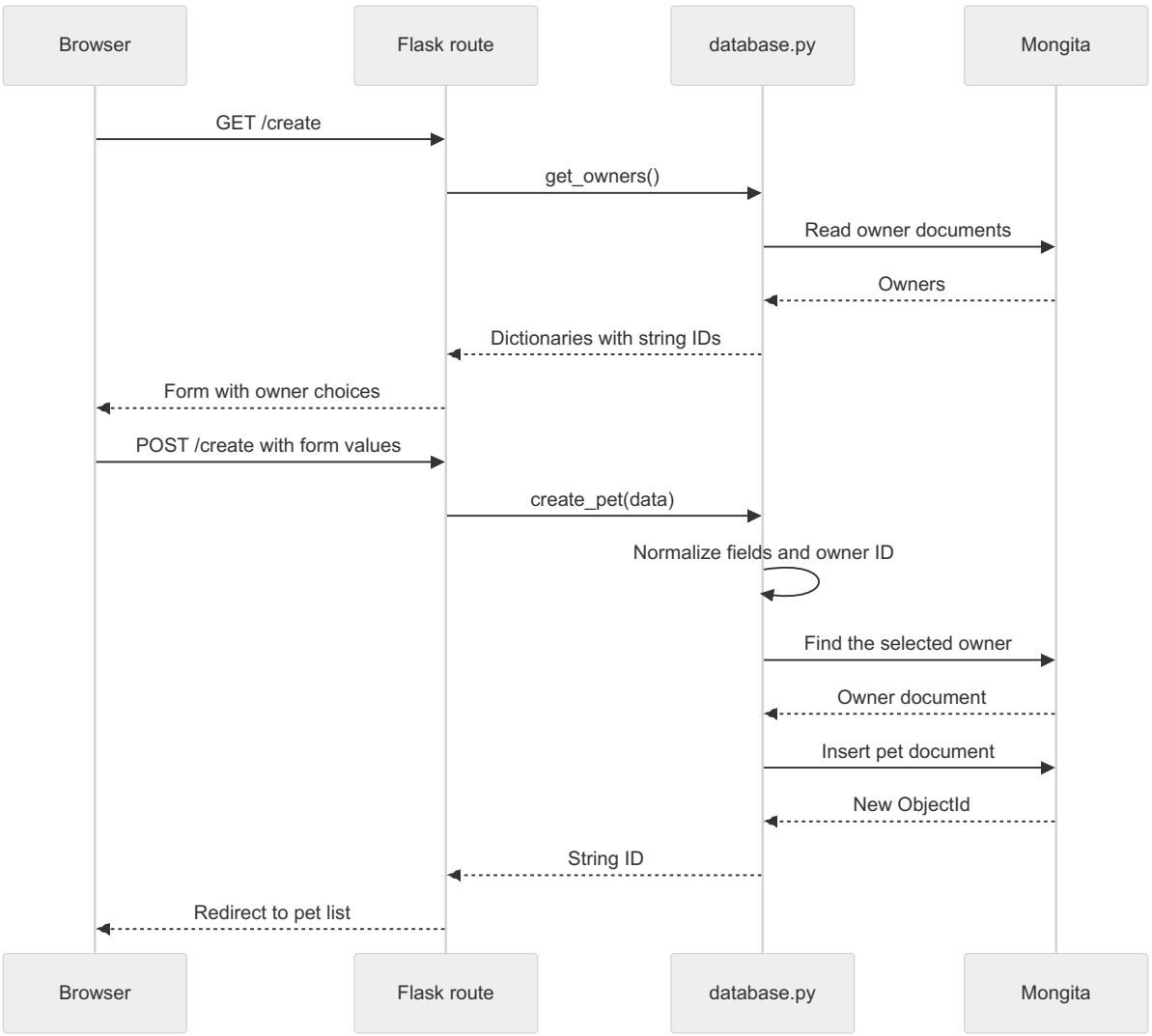
On the way back, `pet_to_dict()` returns ordinary values for the route and template:

```
return {
    "id": str(pet["_id"]),
    "name": pet["name"],
    "type": pet["type"],
    "age": pet["age"],
    "owner_id": str(pet["owner_id"]),
}
```

That keeps database-specific identifier handling inside the layer. A template can build an Update link from `pet["id"]`, and the route uses `/update/<id>` to receive the text.

Follow a Create Request

Loading the create page and submitting it are separate requests. The first obtains the available owners for the dropdown. The second saves the form values.



The route gets the submitted values with `dict(request.form)`. The layer's create operation is short because validation has a separate function:

```
def create_pet(data):
    pet = _normalize_pet_data(data)
    result = pets_collection.insert_one(pet)
    return str(result.inserted_id)
```

`_normalize_pet_data()` requires a name and type, converts the age, and checks the selected owner. For form input, a blank age becomes zero. A negative age or non-numeric text is rejected. Only after these checks does the function insert a document.

The owner check uses the converted identifier:

```
def _require_owner(owner_id):
    object_id = _to_object_id(owner_id, "owner_id")
    owner = owners_collection.find_one({"_id": object_id})
    if owner is None:
        raise ConstraintError(
            "owner_id does not reference an existing owner."
        )
    return object_id
```

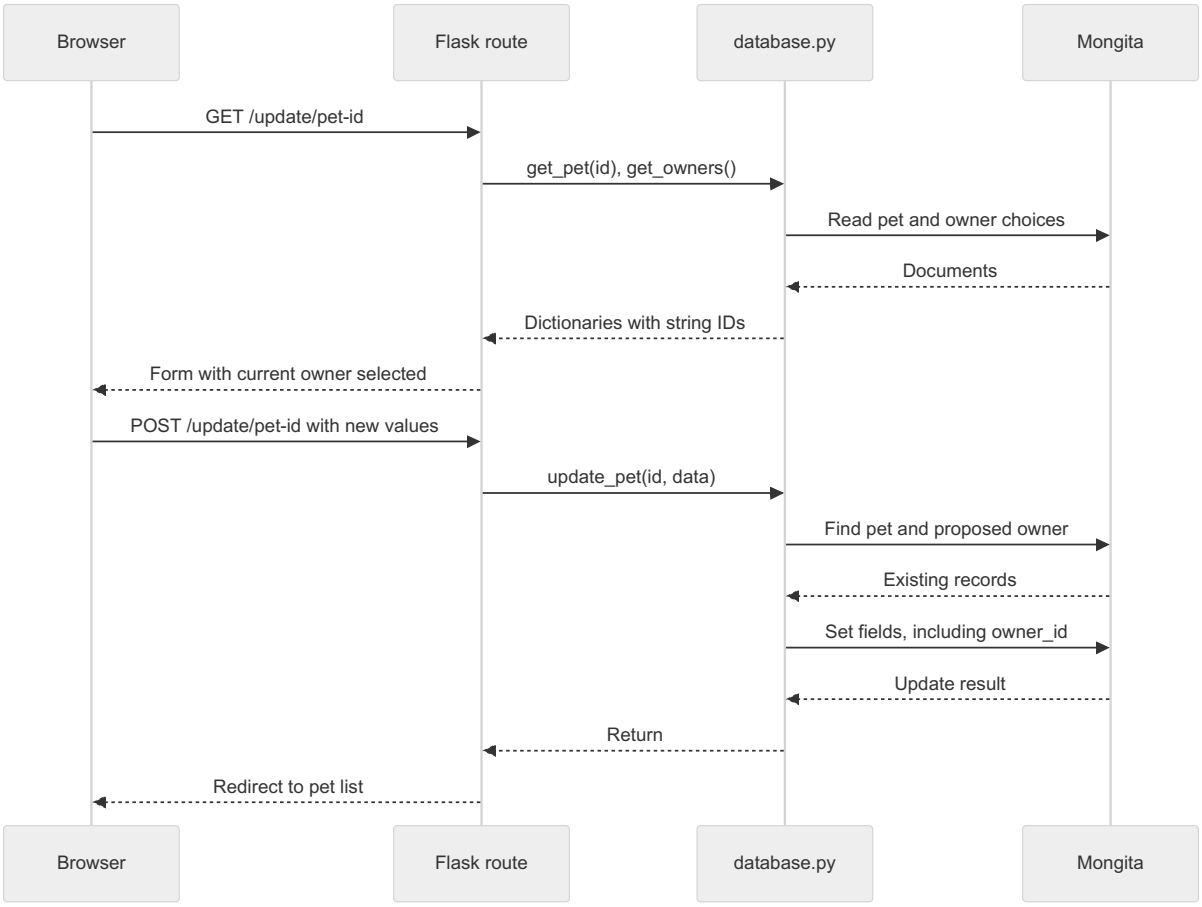
The returned value is the `ObjectId` saved in the pet. The layer has checked that it identifies an owner, not merely that its text looks plausible.

Try This: Submit a pet with an empty name. Then try an age of `-5`, followed by `abc`. Each request should display an error instead of adding a pet. Return to the list after each attempt and check that no extra record appeared.

Load, Edit, and Reassign a Pet

Try This: Open Casey's Update page. Change the age to 4 and select Blair as the owner. Save the form.

The address identifies the pet being changed. The selected dropdown value identifies the new owner. They are separate identifiers with different jobs.



The actual update function is:

```
def update_pet(id, data):
    object_id, _ = _require_existing_pet(id)
    pet = _normalize_pet_data(data)
    pets_collection.update_one({"_id": object_id}, {"$set": pet})
```

The first helper checks that the target pet exists. The second checks the proposed field values and owner. `$set` then changes the supplied fields while leaving the pet's `_id` unchanged.

Value	Before saving	After saving
Pet identifier	Casey's identifier	Same identifier
Age	3	4
Owner reference	Alex's identifier	Blair's identifier
Owner documents	Alex and Blair	Same two owners

Reassignment changes Casey's reference. It does not rename an owner or move an owner document. The list displays the stored owner identifier, so compare it with the identifiers on the owners page. Reopen Casey's Update form and confirm that Blair is selected.

Try This: Attempt another update with age `-5`. Check that Casey still has age 4 and still belongs to Blair. Validation happens before the document update, so the rejected form does not partially save its other changes.

Deletion and Relationship Rules

Deleting a pet removes its document. Deleting an owner requires another question: does any pet still refer to this owner?

```
def delete_owner(id):
    object_id, _ = _require_existing_owner(id)
    pet = pets_collection.find_one({"owner_id": object_id})
    if pet is not None:
        raise ConstraintError(
            "Cannot delete this owner because they have pets. "
            "Please delete their pets first."
        )
    owners_collection.delete_one({"_id": object_id})
```

The message is wrapped here for readability. One matching pet is enough to block the deletion; the function does not need a list of every pet owned by that person.

These checks implement the referential integrity that the relational database handled through foreign-key constraints. In this version, `database.py` checks both sides of the relationship: an owner must exist when we save a pet, and no pets may refer to an owner when we delete that owner. Calls through the layer enforce these rules; direct writes to the collections do not run our Python checks.

Try This: Attempt to delete Blair while Casey belongs to Blair. The app should refuse. Reassign Casey to Alex, then try deleting Blair again. The second deletion should succeed, provided no other pets refer to Blair.

Errors Tell Us What Happened

The layer separates malformed input from a missing record and from a relationship violation. The Flask routes turn those outcomes into Hypertext Transfer Protocol (HTTP) responses.

Situation in this app	Result
Identifier cannot be converted	400: invalid input
Update target does not exist	404: record not found
Owner still has pets	400: relationship rule rejected the request
Successful create or update	302: redirect to the list

A successful save redirects rather than rendering the list directly in response to the form submission. The browser then requests the list page. Flask supplies the request, redirect, and response helpers used by these routes. (3)

Try This: Open `/update/not-an-id` on the running app. Then try `/update/000000000000000000000000`, assuming that identifier is absent from your store. The first request tests identifier syntax; the second tests whether a syntactically valid identifier identifies an existing pet.

Check the Saved State

Try This: Stop Flask with Ctrl+C and restart it with the same command:

```
flask --app app run --debug
```

Reopen the list. Casey's age and owner should still reflect the successful save. Restarting the program recreates Python variables; it does not reset the disk store.

For a direct look at the documents, stop Flask again before opening a separate Python session against the same local store. Use one running application or interactive session at a time for this exercise. (2)

Try This: Still in `chapter-12-mongo-app/code`, run `python`, then enter:

```
import database
from bson.objectid import ObjectId

database.setup_database("pets")
pets = database.get_pets()
print(pets)
casey = next(p for p in pets if p["name"] == "Casey")
print(type(casey["id"]).__name__)
stored = database.pets_collection.find_one(
    {"_id": ObjectId(casey["id"])}
)
print(type(stored["_id"]).__name__)
print(type(stored["owner_id"]).__name__)
print(database.get_owner(casey["owner_id"])["name"])
database.close_connection()
exit()
```

This assumes the Casey record from the browser exercise still exists and has a distinct name. The three type names are `str`, `ObjectId`, and `ObjectId`. After the reassignment exercise, the owner name is Alex. We are looking at the same data through two interfaces: the application's dictionaries and the stored documents.

Practice

1. Create an owner with two pets. Record all three identifiers. Rename the owner and check whether either pet's `owner_id` changes. Explain what the result tells you about names and identity.
2. Create two pets with the same name but different ages. Change one using its Update link. Inspect the stored records and show which identifier selected the changed pet.
3. Extend the pet list to display the owner's name alongside the identifier. Keep the lookup inside `database.py`; return an `owner_name` field for the template. Verify that renaming an owner changes the displayed name without rewriting every pet.
4. For that extension, count how many owner lookups are performed when one owner has many pets. Try loading the owners once into a dictionary keyed by identifier, then use that dictionary while building the pet list. Compare the number of lookups before and after.

The supplied tests provide another way to check the application. **Try This:** From the same `code` directory, after leaving Python:

```
python -m pip install pytest
python -m pytest -q database.py test_database_ci.py
```

The suite exercises the database functions and Flask routes. Its persistence test uses a temporary directory, so it does not replace the records from your browser work.

Considerations

- When would embedding an owner's details inside every pet make reads easier? What would an owner-address change require?
- Which rules belong to the meaning of this application, regardless of whether storage uses tables or documents?
- Why should a route distinguish an invalid identifier from a valid identifier that finds no record?
- If another program writes directly to the collections, which guarantees from our database layer would it need to preserve?

References

1. Wikipedia contributors. "Document-oriented database." https://en.wikipedia.org/wiki/Document-oriented_database
2. Scott Rogowski and contributors. "Mongita." Project README and API summary. https://github.com/scottrogowski/mongita
3. Pallets Projects. "Quickstart." Flask documentation, version 3.1. https://flask.palletsprojects.com/en/stable/quickstart/
4. MongoDB, Inc. "Best Practices for Data Modeling in MongoDB." MongoDB Manual. https://www.mongodb.com/docs/manual/data-modeling/best-practices/
5. MongoDB, Inc. "objectid - Tools for working with MongoDB ObjectIds." PyMongo documentation. https://pymongo.readthedocs.io/en/stable/api/bson/objectid.html